Graph Analytics

Prof. Laura Po

Some materials are taken from "CS224W: Machine Learning with Graphs" course held by prof. Jure Leskovec, Stanford University, http://cs224w.stanford.edu

OBJECTIVES

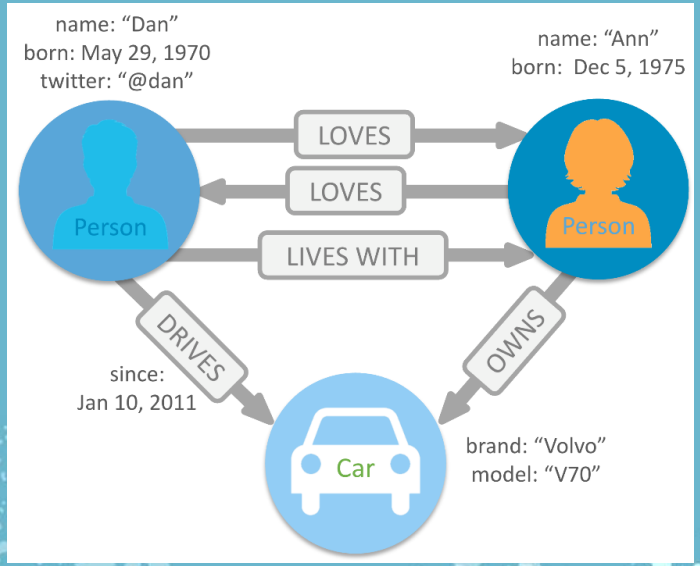
- Provide an overview on graph databases
- Start with the exploration of Neo4J

Graph Database

What is a graph database?

A **graph database** is **a data-storage engine** that combines the basic graph structures of vertices and edges with a persistence technology and a traversal (query) language to create a database optimized for storage and fast retrieval of highly connected data.

Entities and relationships equal importance
Less structurally rigid than relational DB
No limit on the number of relationships a node can have.
Edge/node can have multiple characteristic s which define it.



Challenges of representing multiple varying types of relationships

- Relational databases are poor at representing rich relationships. The relationships in relational databases are **foreign keys**, which are pointers to primary keys in other tables.
- These pointers are not things we can observe and manipulate easily. Instead, the foreign keys are followed (at query time) from one row to another row.

- Graph databases provide excellent tools for moving through relationships in our data. By making the **connections** (edges) as important as the items, graph databases represent these associations as full-fledged constructs of the database that can be easily observed and manipulated.

Advantage of graph db

- Graph databases enhance **developer productivity**.
- Storing data to better represent its real-world counterpart can make it easier for d evelopers to reason over and understand the domain in which they are working. T his allows new team members to get up to speed more quickly on the domain.
- They learn the domain and its database representation simultaneously.

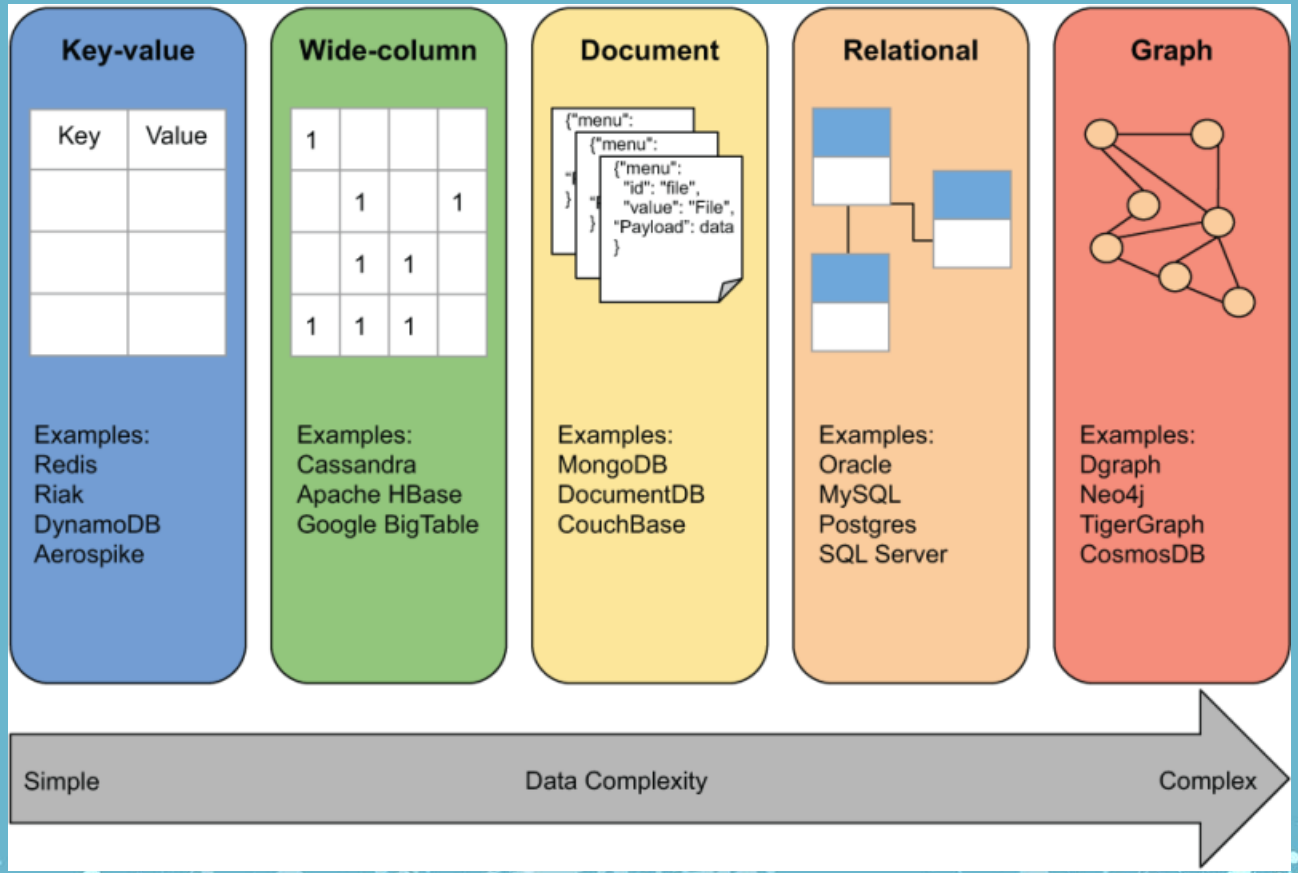
Potential Issues with Graph Databases

- **Security and privacy —** graph databases require more strict implementations of security and acce ss measures. Since graph databases are more oriented toward mapping relationships, that structure ca n also be utilized in ways that could raise privacy concerns, such as revealing a more laid-bare view of a client or customer—and every other potential client or customer to which they are related.

- **Data integrity implications —** Graph databases simplify the ways in which information relates to o ther information. In doing so, shortening or condensing the relationship (as compared to traversing num erous tables in a relational database), it's particularly vital that all data in a graph database is accurate. One improperly aligned relationship can directly lead to incorrect data, unlike in a relational database w here improper data might hit a snag during a nested query, throw an error, and out the issue. So, in usin g graph databases, data integrity is of particularly high importance.

Comparison with other types of databases

NOTE: only the relational databases and graph databases, by default, include the ability to relate entities within the data.



Additional readings

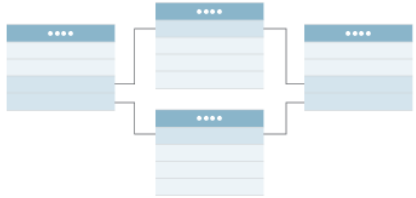
- A.Santra, et al, From base data to knowledge discovery – A life cycle approach – Using multilayer networks, Data & Knowledge Engineering, 141, 2022,https://doi.org/10.1016/j.datak.2022.102058.

- M.Aldwairi et al, Graph-based data management system for efficient information storage, retrieval and processing,Information Processing & Management, 60 (2), 2023, https://doi.org/10.1016/j.ipm.2022.103165

- B. Bender, et al, A proposal for future data organization in enterprise systems—an analysis of established database approaches. Inf Syst E-Bus Manage 20, 441–494 (2022). https://doi.org/10.1007/s10257-022-00555-6

Graph vs relational DB

Graph database vs. relational database

	Graph database	Relational database
FORMAT	Nodes and edges	Tables with rows and columns
RELATIONSHIPS	Considered data, represented by edges between nodes	Related across tables, established using foreign keys between tables
COMPLEX QUERIES	Run quickly and do not require joins	Require complex joins between tables
TOP USE CASES	Relationship-heavy use cases, including fraud detection and recommendation engines	Transaction-focused use cases, including online transactions and accounting
EXAMPLE		

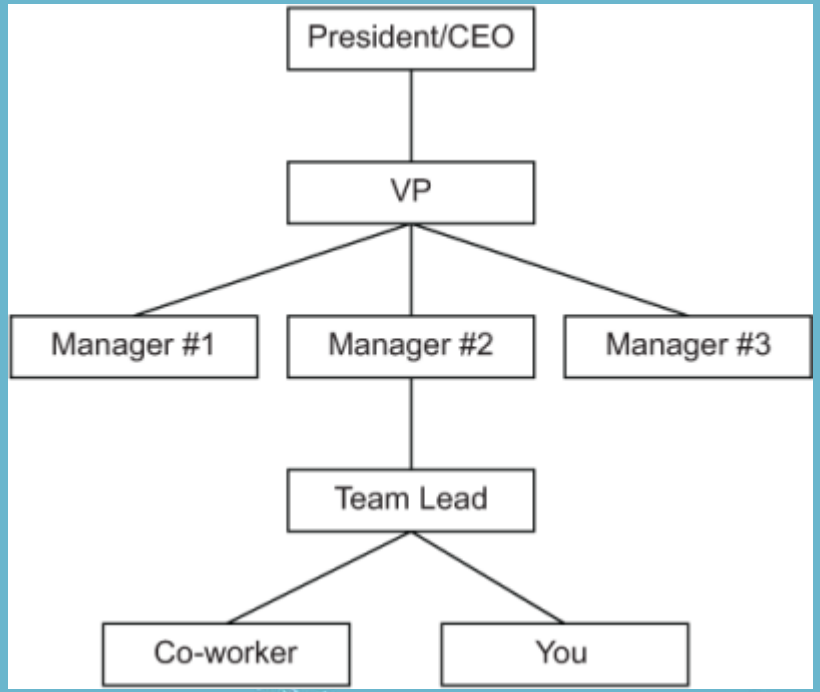
Graph db vs Relational db

There are three areas where graph databases provide a simpler, more efficient solution than using a relational database:

- **Recursive queries** (for example, an organization's employee reporting hierarchy, or org chart)
- **Different result types** (for example, orders and products repo rting example)
- **Paths** (for example, a river-crossing puzzle)

Recursive queries (example)

Recursive queries are executed multiple times in succession, repeatedly calling themselves until they reach some escape or terminating condition.



Model hierarchies in relational db

```
CREATE TABLE org_chart (
manager_employee_id   SM
ALLINT NULL,
employee_name
VARCHAR(20)
NOT NULL
);
```

employee_id	Manager_employee_id	employee_name
1	3	You
2	3	Co-worker
3	4	Team Lead
4	5	Manager #2
5	8	VP
6	5	Manager #1
7	5	Manager #3
8	NULL	President/CEO

Recursive queries in relational db

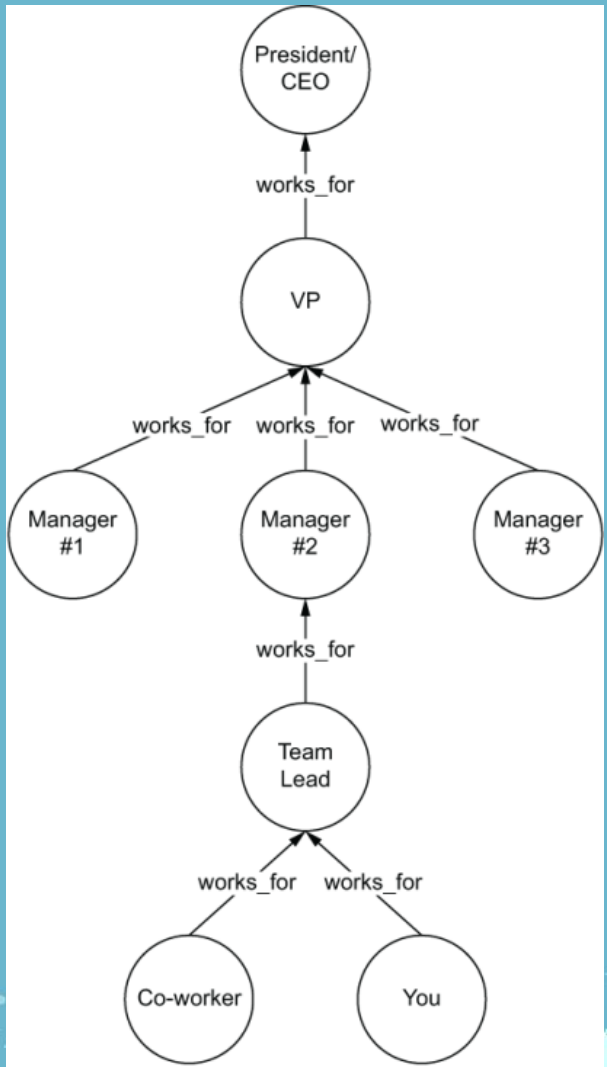
```
WITH RECURSIVE org AS (
    SELECT employee_id, manager_employee_id,
          employee_name, 1 AS level
    FROM org_chart
  UNION
    SELECT e.employee_id, e.manager_employee_id,
          e.employee_name, m.level + 1 AS level
    FROM org_chart AS e
      INNER JOIN org AS m
ON e.manager_employee_id = m.employee_id
  )

SELECT employee_id, manager_employee_id, employee_name
FROM org
ORDER BY level ASC;
```

Model hierarchies in graph db

Traversal query

```
g.V().
  repeat(
'works_for')
  ).path().next()
```



Different result types

In a Relational DB

- Have you ever needed to return several different data types from a database, all within a single result set?
- For instance, let's say that we have an order-processing system and we want to return not only the or der information but also the product information.

Orders

id	name	address
1	John Smith	123 Main. St
2	Jane Right	643 Park St.

Products

id	product_name	cost
123	widget 1	5.95
234	widget 2	10.76

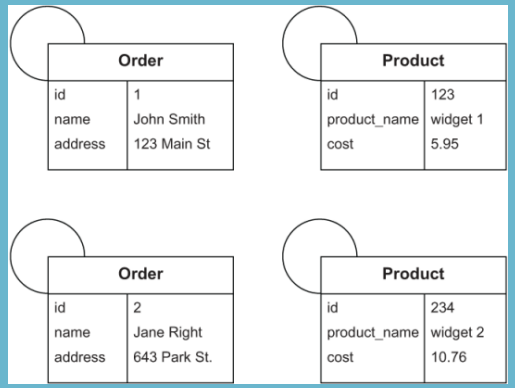
```
SELECT id,
       name,
       address,
       null AS product_name,
       null AS cost,
  'Order' AS object_type
FROM Orders
UNION
SELECT id,
       null AS name,
       null AS address,
       product_name,
       cost,
       'Product' AS object_type
FROM Products;
```

id	Name	Address	product_name	cost	object_type
1	John Smith	123 Main St	<null>	<null>	Order
2	Jane Right	234 Park St	<null>	<null>	Order
123	<null>	<null>	widget 1	5.95	Product
234	<null>	<null>	widget 2	10.76	Product

Different result types

In a Graph DB

How that same data appears in a graph database



we can write a graph traversal to return both product and order data. In this example, a graph database returns these results:

```
gremlin> g.V().valueMap(true)
==>[label:order, address:[123 Main St], name:[John Smith], id:1]
==>[label:order, address:[234 Park St], name:[Jane Right], id:2]
==>[label:product, cost:[10.76], id:234, product_name:[widget 2]]
==>[label:product, cost:[5.95], id:123, product_name:[widget 1]]
```

Path (example)

- A path is the sequence of vertices and edges that describe how the traversal moved through the graph; for example, in Google or Apple Maps, a set of directions between two locations.
- The ability to return how two objects are connected to each other from within the database is a fe ature unique to graph databases.

River crossing puzzle

We have a fox, a goose, and a bag of barley that must be transported across a river by a farmer on a boat.

Constraints:
The boat can only carry one item in addition to the farmer on each trip.
The farmer must go on each trip.
The fox cannot be left alone with the goose or it will eat it.
The goose cannot be left alone with the barley or it will eat it.

Modelling the status

Let’s start by modeling the initial state of our system as a vertex in our graph.

Different characters represents part of the problem:

T (the boat and the farmer)

G (the goose)

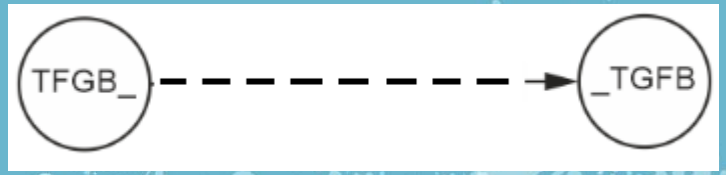
F (the fox)

B (the barley)

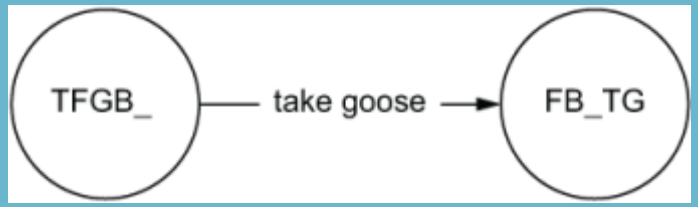
_ (the river)

We will use the vertices representing possible states, we use edges to show how we transition from one st ate to the next.

*TGFB_* vertex encodes the initial state when the boat (T), the goose (G), the fox (F), and the barley (B) ar e all on one side of the river (_). Our goal is to achieve a final state *_TGFB* vertex where they are all on th e other side of the river.

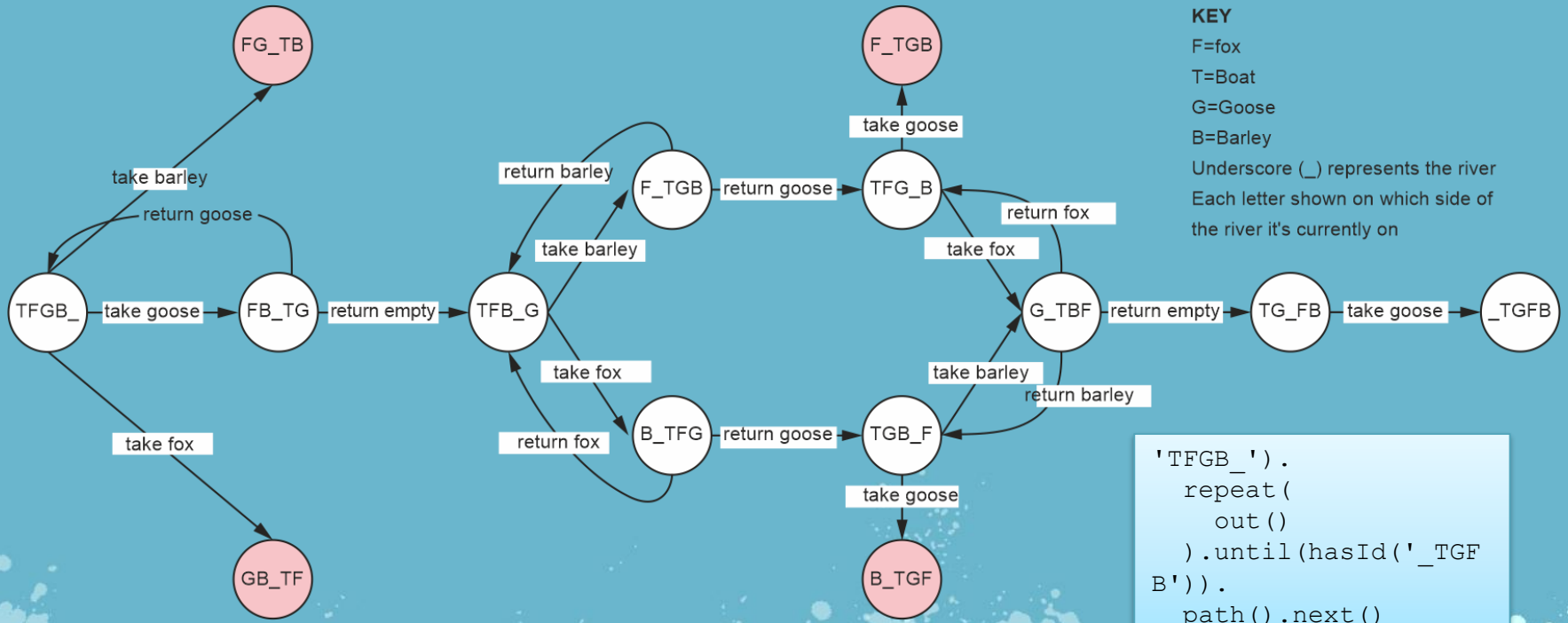


Change of the status



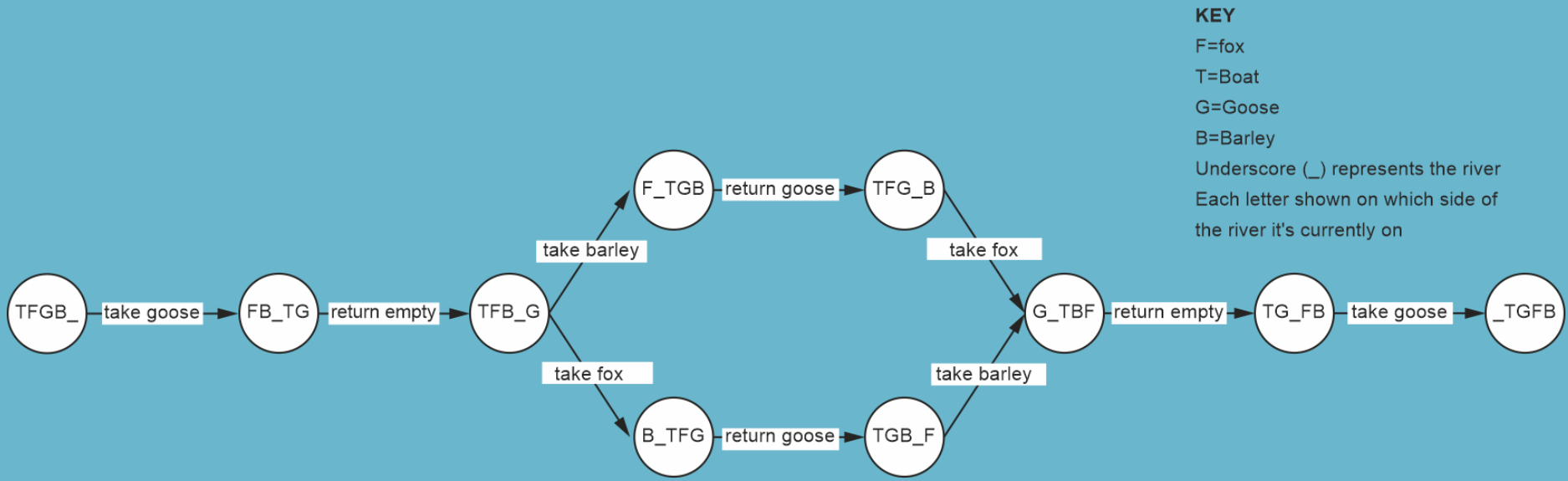
Full graph using a pathfinding algorithm

Modeling all the potential options as a representation of these states (vertices) and state changes (edges).




```
'TFGB_').
  repeat(
    out()
  ).until(hasId('_TGF
B')).
  path().next()
```

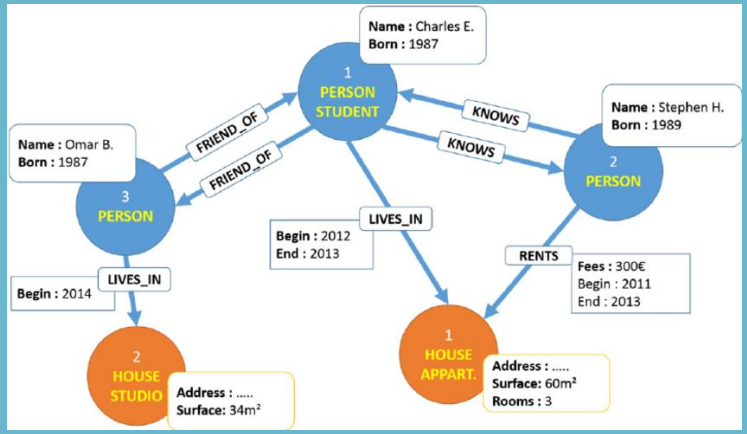
Only the valid states



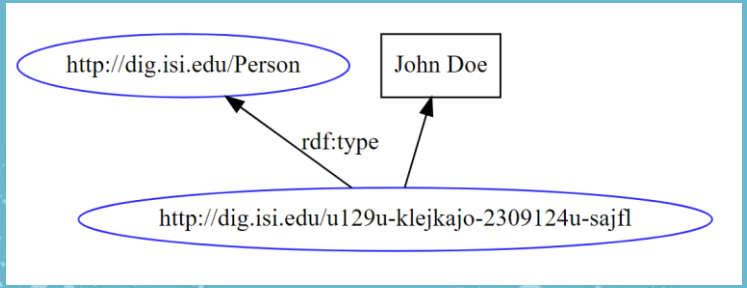
Graph DB

Types of graph databases

- **Property Graph Database**: [most common] it stores data as nodes and edges, and allows for pr operties to be assigned to both.

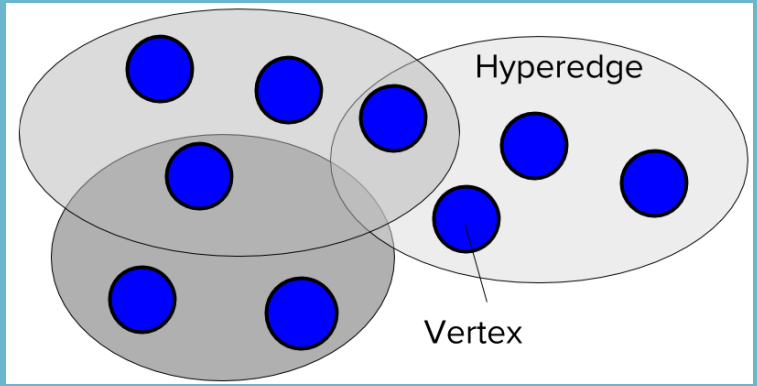


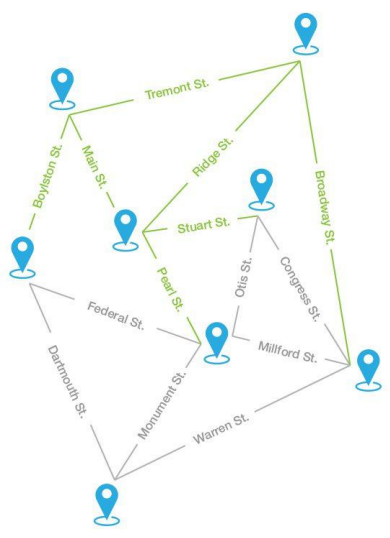
- **Resource Description Framework (RDF) Graph Database**: It uses the RDF model, whic h represents data as triples consisting of a subject , predicate, and object.



Types of graph databases

- **Hypergraph Database**: It stores nodes and hyperedges; hyperedges connect any number of nodes at once (in contrast to multi graphs that connect 2 nodes).

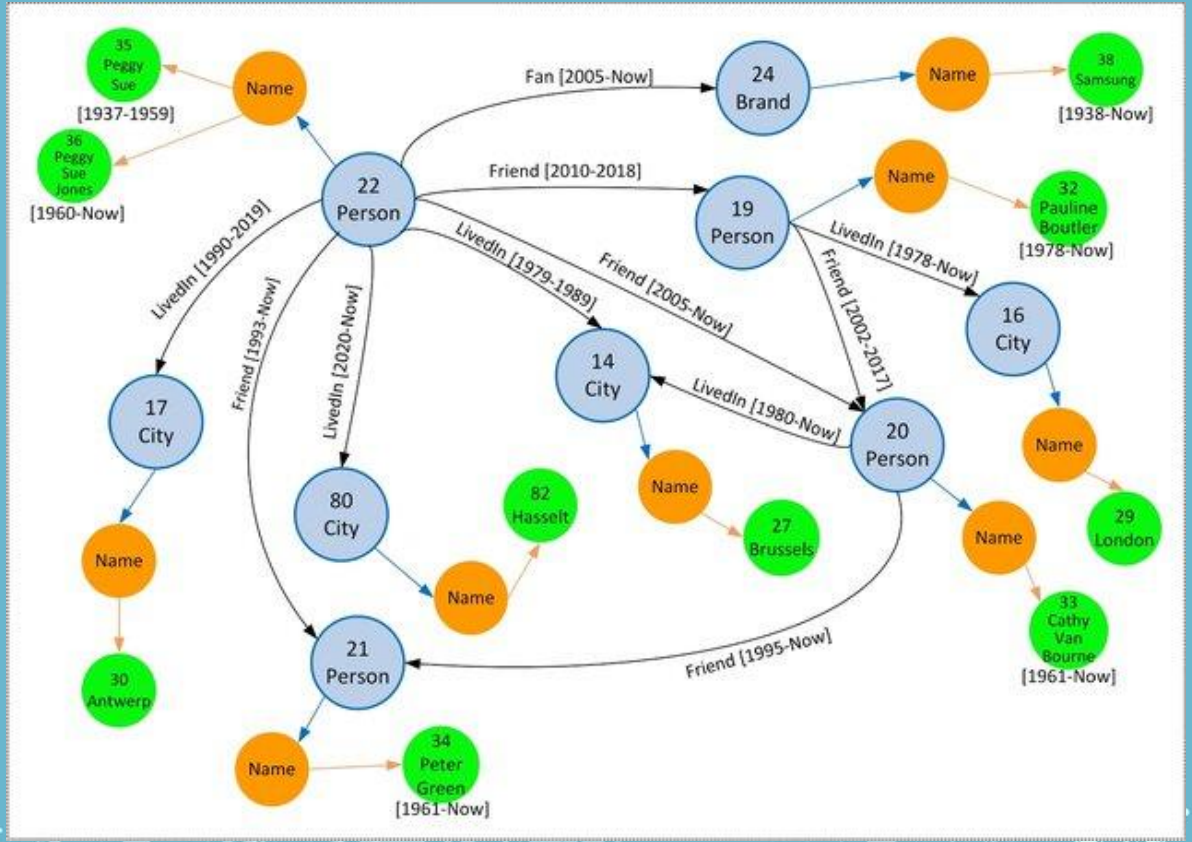




Spatial Graph Database: It is designed f or storing and querying spatial data, such as maps, GPS coordinates, and geographical fe atures. Nodes represent points/locations, an d edges represent relationships between tho se points. They support spatial indexing and spatial querying.

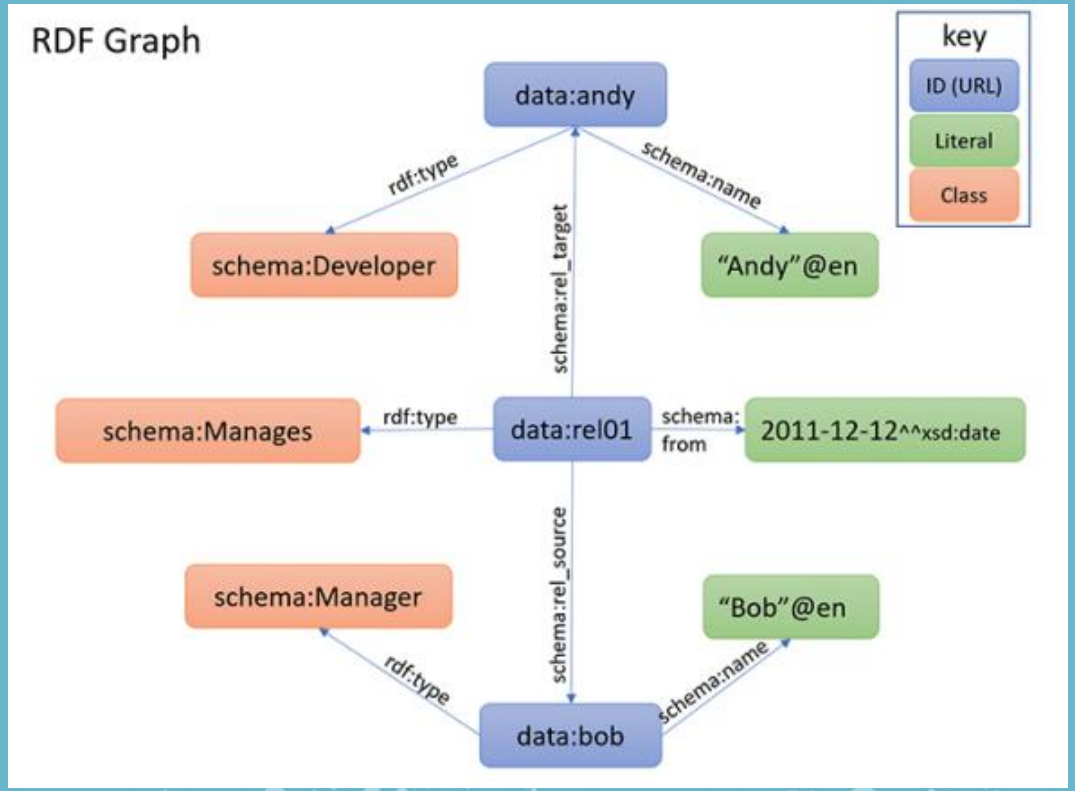
Types of graph databases

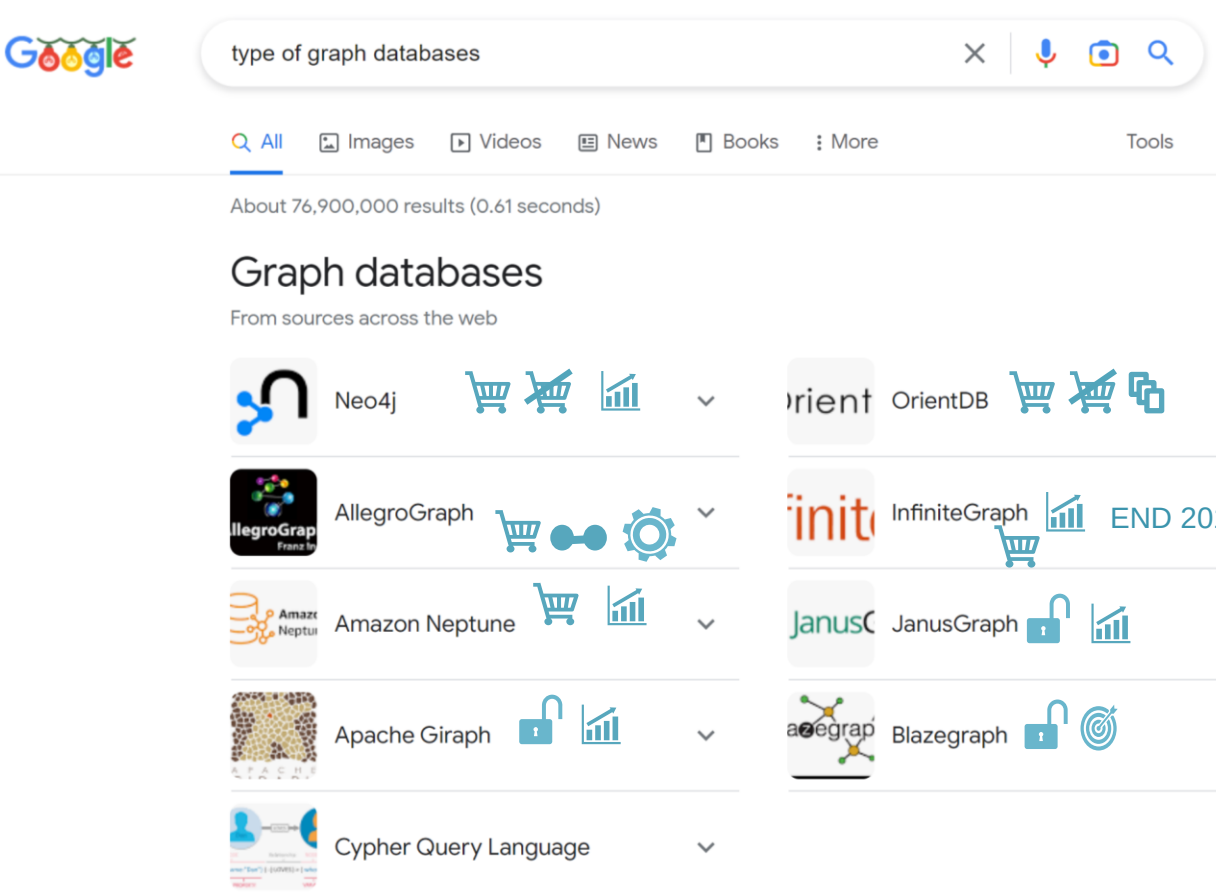
- **Temporal Graph Database**: It is designed for storing and que rying data with a temporal dimen sion, such as time series data or event logs. Nodes represent enti ties, and edges represent relatio nships between those entities at specific points in time.



Types of graph databases

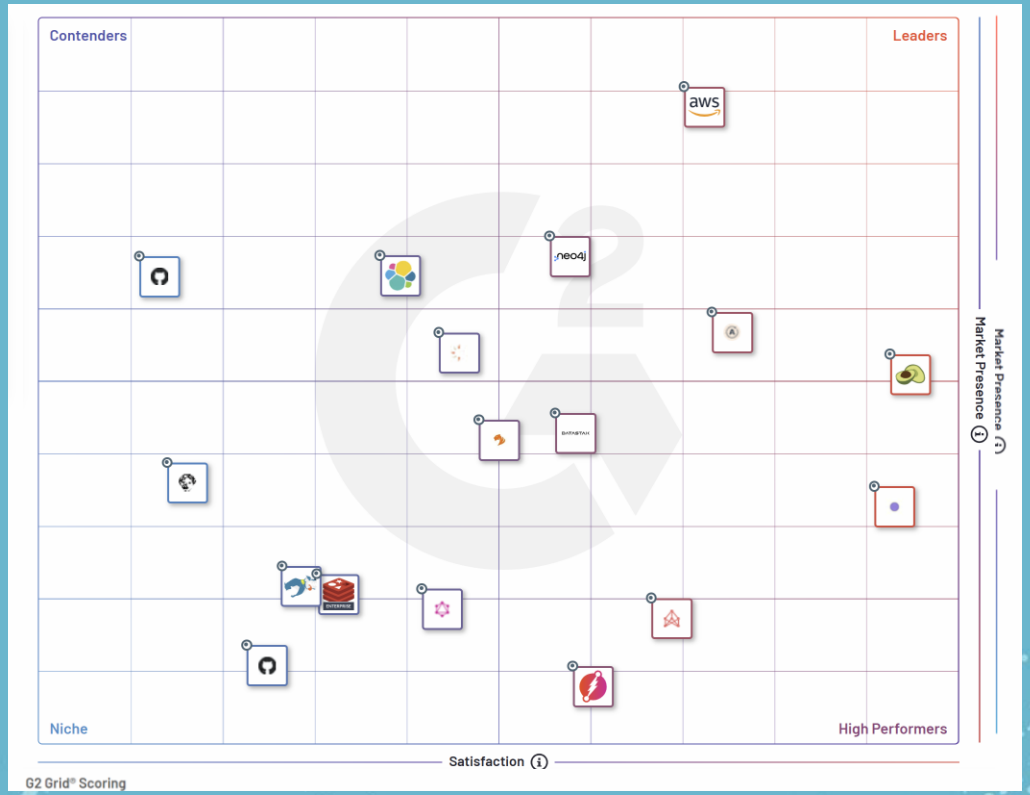
- **Property-Graph/ Triplestore Hybrid**: A graph database that c ombines features of both property graph and RDF triplestores.





commercial edition
community edition
RDF data
reasoning
scalable
open source
high perf. Indexing
multimodel

G2 Grid® - top Graph Databases products



If you want to explore how platform are rank ed go to G2Grid

https://www.g2.com/categories/graph-databases#grid

Tools

HOW TO MODEL GRAPH DB

Neo4j is a graph database management system that is designed to store and process highly connected data. Neo4j uses a property graph data model, which means that each node and edge in the graph can have properties associated with it. Neo4j is optimized for high performance and scalability, and it has a number of built-in features that make it easy to work with graph data.

TinkerPop: TinkerPop is a graph computing framework that provides a set of interfaces and APIs for working with graph data in a vendor-agnostic way. It provides a standard set of operations and algorithms that can be used across different graph databases, and supports a variety of programming languages, including Java, Python, and .NET.

HOW TO QUERY GRAPH DB

GraphQL is a declarative query language for APIs, can be used with a variety of database systems (relational, document and graph db).
It supports a range of advanced features, such as query batching, pa gination, and introspection, and is designed to be highly flexible and adaptable to different types of applications.

Cypher: Cypher is a declarative query language that is designed specifically for **Neo4j**. It provides a simple, expressive syntax for querying and manipulating graph data, and supports a range of operations and functions that are optimized for Neo4j's storage.

Gremlin is a functional, declarative graph traversal languag e that is also part of the TinkerPop framework.
It provides a set of operators and functions for querying and manipulating graph data, and supports a range of graph algorithms and traversal strategies.

HOW TO QUERY GRAPH DB

https://www.gqlstandards.org/

GQL (**Graph Query Language**) is a proposed standard graph query language.
In September 2019 a proposal for a project to create a new standard graph query language (ISO/IEC 39075:2024 Information technology — Database languages — GQL) was approved by a national standards bodies which are members of ISO/IEC Joint Technical Committee.
In April 2024, the GQL Standard has been published.

https://www.iso.org/standard/76120.html

https://jtc1info.org/wp-content/uploads/2024/04/2024-Article-39075-Database-Language-GQL.docx.pdf


neo4j

Components of the Neo4j Graph Data Platform

https://neo4j.com/docs/getting-started/current/get-started-with-neo4j/graph-platform/

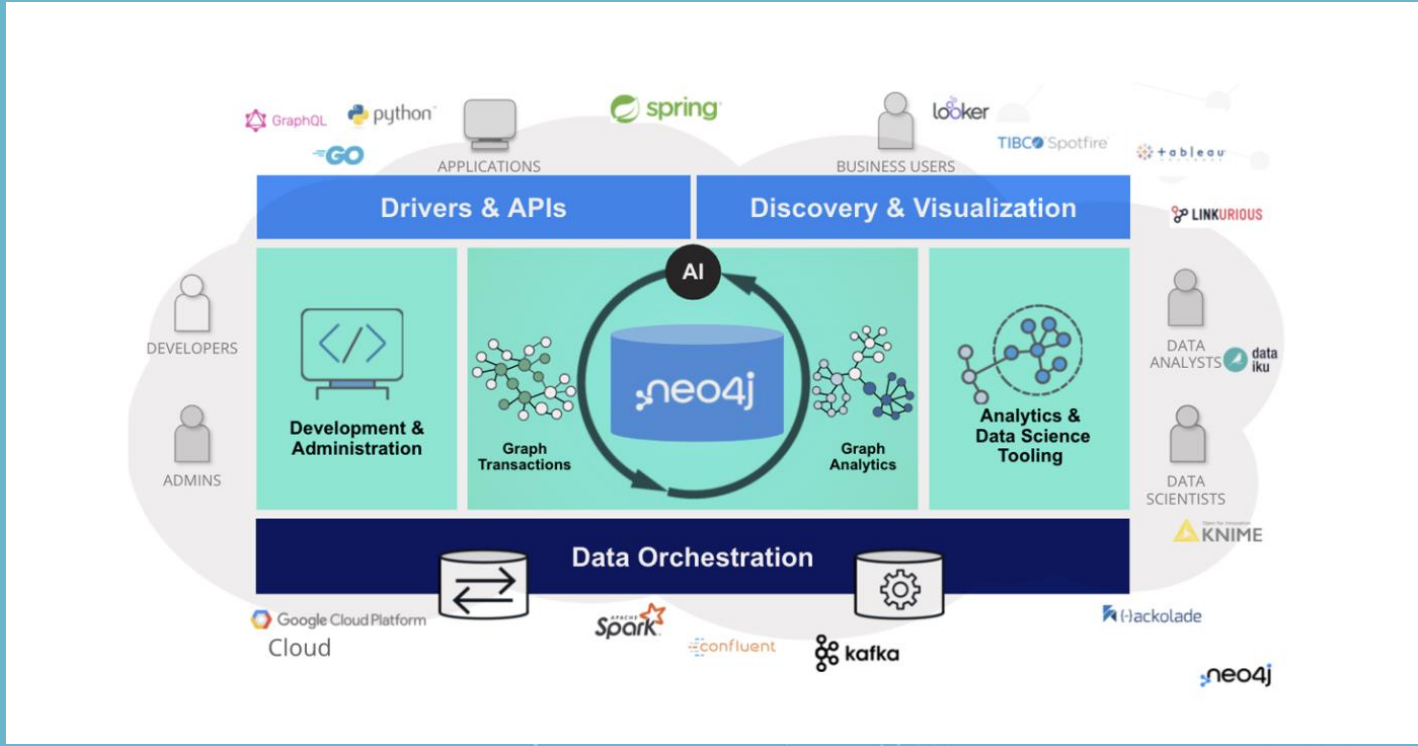
For business users

Cloud service

Local service

Neo4j Graph Data Science (GDS)

A full stack Graph application

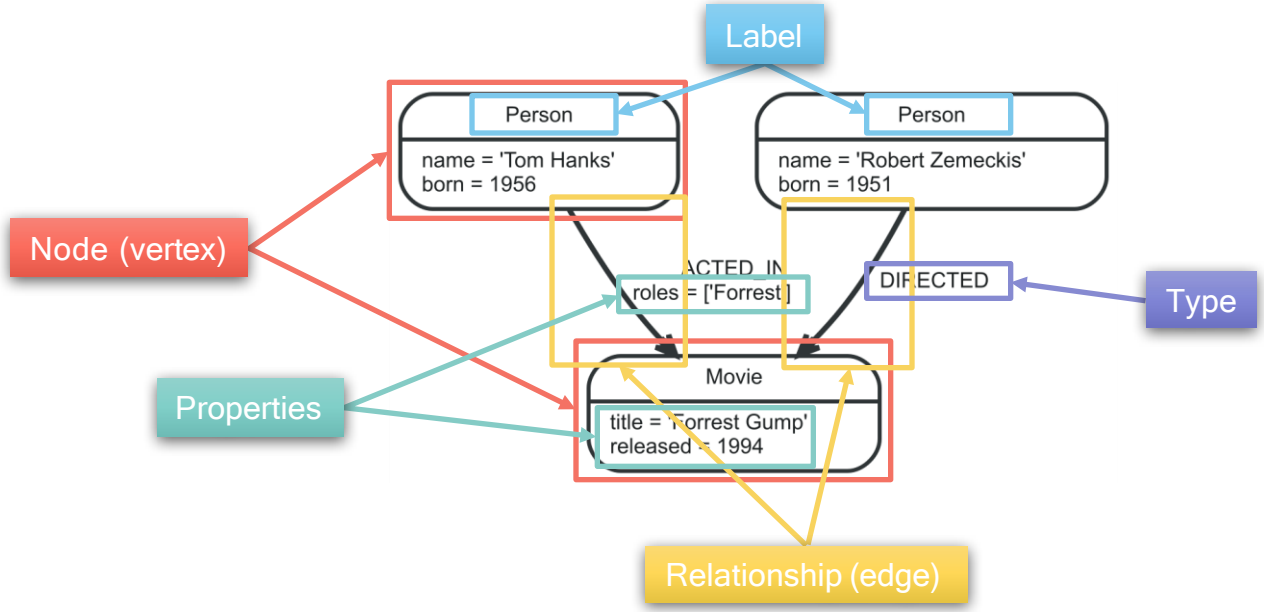


https://www.youtube.com/watch?v=T6L9EoBy8Zk

Neo4J Terminology & Syntax

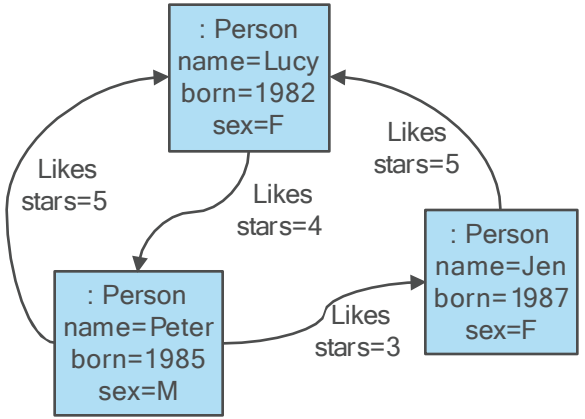
Some slides are taken from the "Graph Analytics" course held at the Sixth European Business Intelligence & Big Data Summer School 2016 by the Dresden Database sytem Group of the TUD

Neo4j Terminology



https://neo4j.com/docs/

Match



MATCH-CLAUSE

- Primary way of getting data from a Neo4j database
- Allows you to specify the patterns
- Named pattern element, e.g. (p:Person), will be bound to the match instance
- Query can have multiple MATCH-clauses

RETURN-CLAUSE

- Projects to the result set
- Allows projection to nodes, edges, and properties

```
MATCH (p:Person)-[:Likes]->(f:Person)
RETURN p.name, f.sex
```

p.name	f.sex
Lucy	M
Peter	F
Jen	F
Peter	F

```
MATCH (p:Person)-[:Likes]->(:Person) -[:Likes]->(fof:Person)
RETURN p.name, fof.name
```

p.name	fof.name
Lucy	Jen
Peter	Lucy
Peter	Peter
Jen	Peter
Lucy	Lucy

Pattern Syntax

Vertex pattern

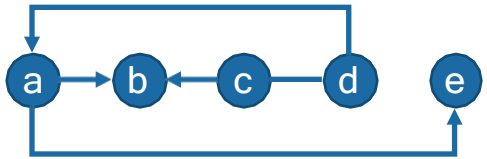
▪ ()	unidentified vertex
▪ (matrix)	vertex identified by variable *matrix*
▪ (:Movie)	unidentified vertex with label *Movie*
▪ (matrix:Movie:Action)	vertex with labels *Movie* and *Action* identified by variable *matrix*
▪ (matrix:Movie {title: "The Matrix"})	+ property *title* equal the string “The Matrix”
▪ (matrix:Movie {title: "The Matrix", released: 1997})	+ property *released* equal the integer *1997*

Edge pattern

▪ -->	unidentified edge
▪ -[role]->	edge identified by variable *role*
▪ -[:ACTED_IN]->	unidentified edge with label *ACTED_IN*
▪ -[role:ACTED_IN]->	edge with label *ACTED_IN* identified by variable *role*
▪ -[role:ACTED_IN {roles: ["Neo"]}]->	+ property *roles* contains the string “Neo”

Pattern Syntax

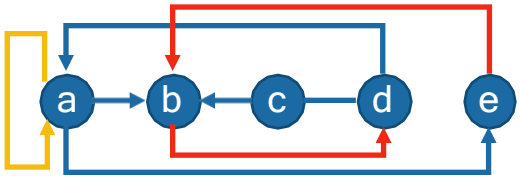
PATH PATTERNS



- String of alternating vertex pattern and edge pattern
- Starting and ending with a vertex pattern
- (a)-->(b)<--(c)--(d)-->(a)-->(e)
- (keanu:Person:Actor {name: "Keanu Reeves"})-[role:ACTED_IN {roles: ["Neo"]}]-> (matrix:Movie {title: "The Matrix"})

GRAPH PATTERNS

- One or multiple path patterns
- Path patterns should have at least one shared variable
- Without shared variable graph pattern is disconnected
  - Results in a cross-product of the results for connected sub patterns
  - Quadrating blow up in result size and computational complexity
- (a)-->(b)<--(c)--(d)-->(a)-->(e), (e)-->(b)-->(d), (a)-->(a)



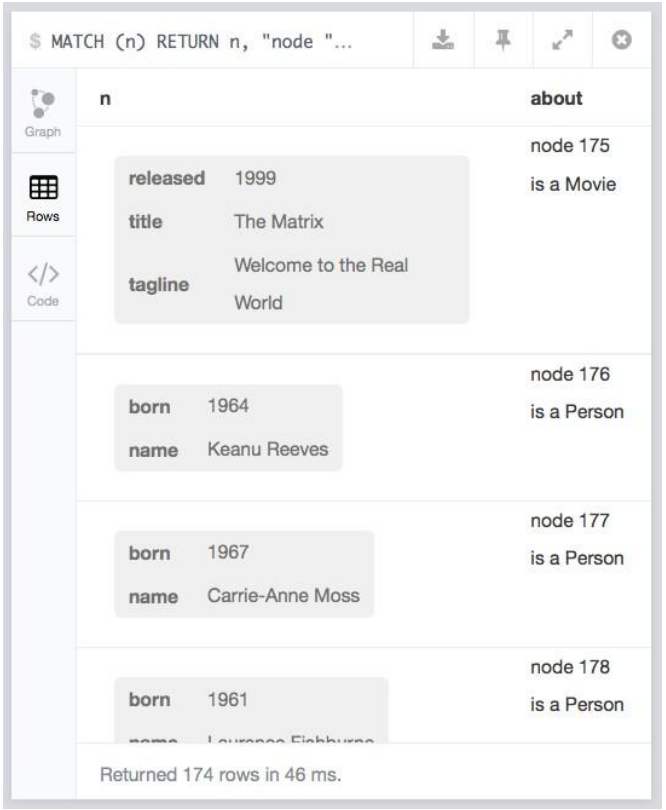
Return

RETURN-CLAUSE

- Defines what to include in the query result set
- Comparable with relational projection
- Only once per query
- Allows to return nodes, edges, properties, or any expressions
- Column can be rename using AS <new name>

EXAMPLE

- MATCH (n)
  RETURN n, "node " + id(n) +" is " +
  CASE WHEN n.title IS NOT NULL THEN "a Movie"
  WHEN EXISTS(n.name) THEN "a Person"
  ELSE "something unknown"
  END AS about



Optional Match & Where

OPTIONAL MATCH-CLAUSE

- Matches patterns against your graph database, just like MATCH
- Matches the complete pattern or not
- If no matches are found, OPTIONAL MATCH will use NULLs as bindings
- Like relational outer join
- Example:

```
MATCH (a:Movie)
OPTIONAL MATCH (a)<-[:WROTE]-(x)
RETURN a.title, x.name
```

$ MATCH (a:Movie) OPTIO...

Rows

Code

a.title	x.name
The Matrix	null
The Matrix Reloaded	null
The Matrix Revolutions	null
The Devil's Advocate	null
A Few Good Men	Aaron Sorkin
Top Gun	Jim Cash
Jerry Maguire	Cameron Crowe
Stand By Me	null
As Good as It Gets	null
What Dreams May Come	null
Snow Falling on Cedars	null
You've Got Mail	null
Sleepless in Seattle	null
Joe Versus the Volcano	null
When Harry Met Sally	Nora Ephron
That Thing You Do	null

Returned 41 rows in 40 ms.

WHERE

- After an (OPTIONAL) MATCH, it adds constraints to the (optional) match
- WHERE becomes part of the pattern
- After a WITH, it just filters the result
- Syntax: WHERE <expression>
- Example:

```
MATCH (n)
WHERE n.name = 'Peter' XOR (n.age < 30 AND n.name = 'Tobias')
                OR NOT (n.name ~= 'Tob.*' OR n.name CONTAINS 'ete')
RETURN n
```

Matching Paths

Variable length path patterns

- Repetitive edge types can be expressed by specifying a length with lower and upper bounds
- Example: (a)-[:x*2]->(b) is equal to (a)-[:x]->()-[:x]->(b)
- More examples:
  (a)-[*3..5]->(b)
  (a)-[*3..]->(b)
  (a)-[*..5]->(b)
  (a)-[*]->(b)
- Complete example:
  MATCH (me)-[:KNOWS*1..2]-(remote_friend)
  WHERE me.name = "Filipa" RETURN remote_friend.name
- Matches unique paths (relationship uniquness), not unique reachable nodes!!!
- Particularly the unbounded [*] easily matches larger numbers of paths -> exponential blowup!!!

Path variables

- Assign matched paths to variable or further processing
- Example: p = ((a)-[*3..5]->(b))

Matching Shortest Paths

SHORTEST PATHS

- Path between two nodes with minimum number of edges
- Apply the shortestPath/allShortestPath function to a path pattern to match single/all shortest paths
- Additional filter predicates can be given with WHERE clause
  - Universal (NONE/ALL) predicates can be evaluated during shortest path search
  - Other predication can be evaluated only after shortest path has been discovered
- Fast evaluation algorithm
  - Bidirectional BFS
  - Standard for paths without additional predicates and path with universal predicates
- Slow evaluation algorithm
  - DFS
  - Fallback for paths with non-universal predicates
- Example (fast evaluation):

```
MATCH (m { name:"Martin Sheen" }),(o { name:"Oliver Stone" }), p = shortestPath((m)-[*..15]-(o))
WHERE NONE(r IN rels(p) WHERE type(r)= "FATHER") RETURN p
```

- Example (fast evaluation):

```
MATCH (m { name:"Martin Sheen" }),(o { name:"Oliver Stone" }), p = shortestPath((m)-[*..15]-(o))
WHERE length(p) > 1 RETURN p
```

Max. 15 hops

Aggregation

IN RETURN-CLAUSE

- Implicit group by
  - Expressions without an aggregation function will grouping keys
  - Expressions with an aggregation function will produce aggregates
- DISTINCT within the aggregation function removes duplicates in a group before the aggregation
- Aggregation function: COUNT, SUM, AVG, MIN, MAX, STDEV, STDEVP, PERCENTILEDISC, PERCENTILECONT, and COLLECT – collects all the values into a list

IN WITH-CLAUSE

- Like a process pipe
- Chains query parts together, piping the results from one to be used as starting points in the next
- Like RETURN, WITH defines – including aggregation – the output before it is passed on
- Allows to
  - Filter on aggregates
  - Aggregation of aggregates
  - Limit search space based on order of properties or aggregates

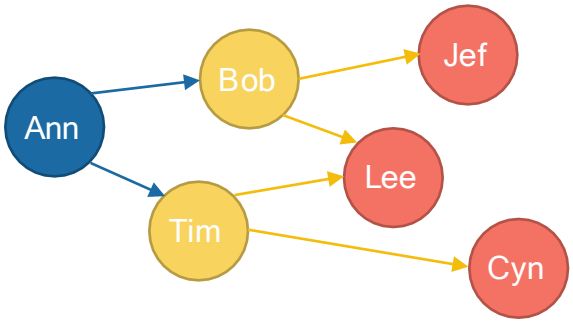
Aggregation

IN RETURN-CLAUSE

- Implicit group by
  - Expressions without an aggregation function will be group keys
  - Expressions with an aggregation function will produce aggregates
- DISTINCT within the aggregation function removes duplicates in a group before the aggregation
- Aggregation function: COUNT, SUM, AVG, MIN, MAX, STDEV, STDEVP, PERCENTILEDISC, PERCENTILECONT, and COLLECT - collects all the values into a list
- Example:

```
MATCH (me:Person {name:'Ann'})-->(friend:Person)-->(friend_of_friend:Person)
RETURN me.name, count(DISTINCT friend_of_friend), count(friend_of_friend)
```

group key — aggregates



Result

me	COUNT DISTINCT	COUNT
Ann	3	4

IN WITH-CLAUSE

- See next slide

Query Composition

WITH-CLAUSE

- Like a process pipe
- Chains query parts together, piping the results from one to be used as starting points in the next
- Like RETURN, WITH defines - including aggregation - the output before it is passed on

- Filter on aggregates
  Example: Soccer team on average younger than 25
  ```
  MATCH (p)-[:PLAYS]->(t) WITH t, AVG(p.age) AS a WHERE a < 25 RETURN t
  ```

- Aggregation of aggregates
  Example: Average age of the youngest player in each team
  ```
  MATCH (p)-[:PLAYS]->(t) WITH t, MIN(p.age) AS a RETURN AVG(a)
  ```

- Limit search space based on order of properties or aggregates
  Example: Friends of five best friends
  ```
  MATCH (p)-[f:FRIENDS]->(p2)
  WITH f,p2 ORDER BY f.rating DESC LIMIT 5
  MATCH (p2)-[f:FRIENDS]->(p3) RETURN DISTINCT p3
  ```

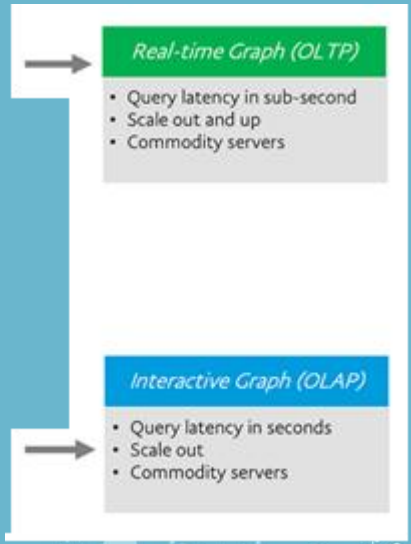
Graph Landscape

A High-Level View of the Graph Space

Numerous projects and products for managing, processing, and analyzing graphs have exploded onto the scene in recent years. The sheer number of technologies makes it difficult to keep track of these tools and how they differ.

We can divide the **graph space** into two parts:

1. Technologies used primarily for transactional online graph pers istence, typically accessed directly in real time called **graph da tabases.** Equivalent of OLTP databases.
2. Technologies used primarily for offline graph analytics, typicall y performed as a series of batch steps, called **graph compute engines**. Similar to data mining and online analytical processin g (OLAP).



Feature	Graph Databases (OLTP)	Graph Compute Engines (OLAP)
Primary Use	Real-time queries	Batch analytics
Access Pattern	Direct, frequent queries	Offline, large-scale computation
Performance Goal	Low-latency, fast lookup	High-throughput, deep analysis
Example Queries	"Who are my friends of friends?"	"Find communities in a social network"
Examples	Neo4j, Amazon Neptune	Apache Giraph, Google Pregel, Spark GraphX

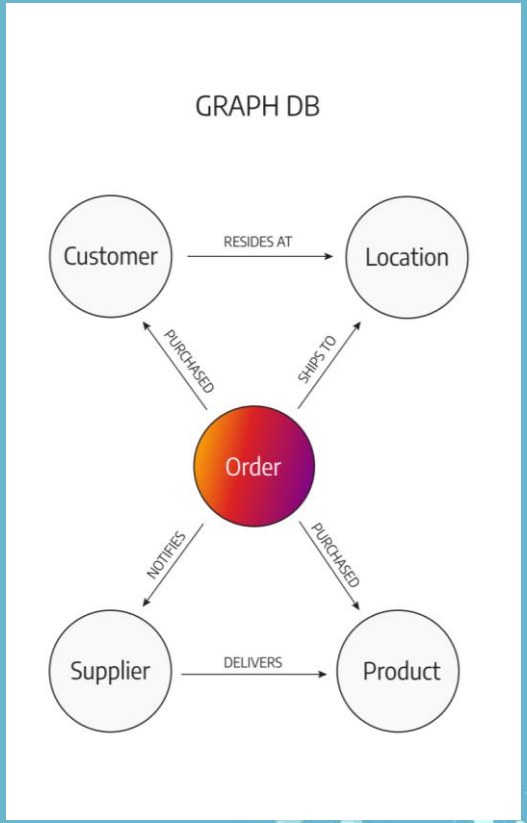
Graph database management system

A *graph database management system* (*graph database*) is an online database management system with Create, Read, Update, and Delete (CRUD) methods that expose a graph data model

Graph databases are generally built for use with transactional (OLTP) systems - they are normally optimized for transactional performance

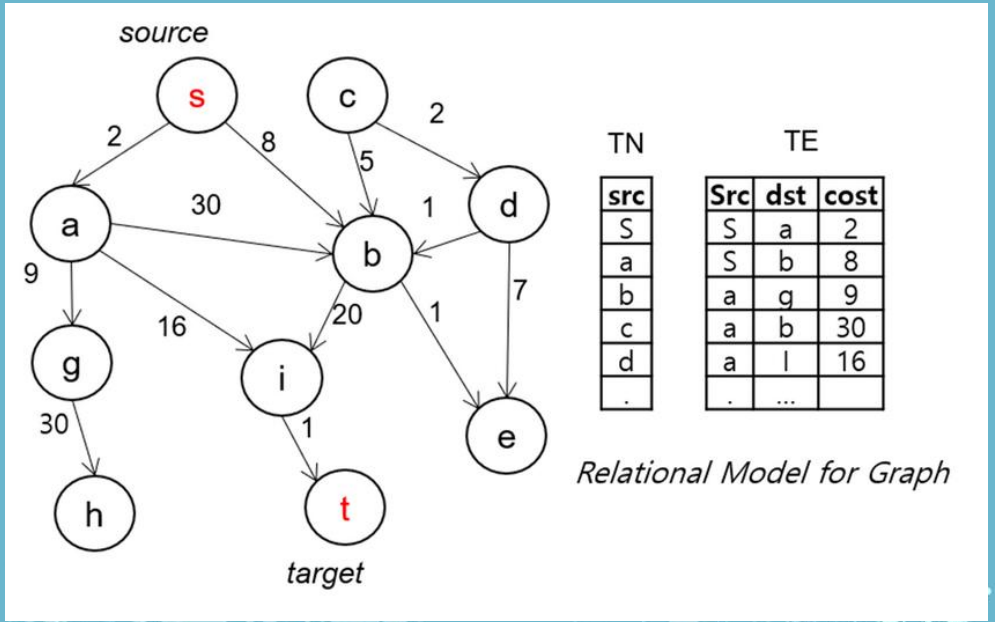
There are two properties of graph databases we should consider when investigating graph database technologies:

- the storage
- the processing engine



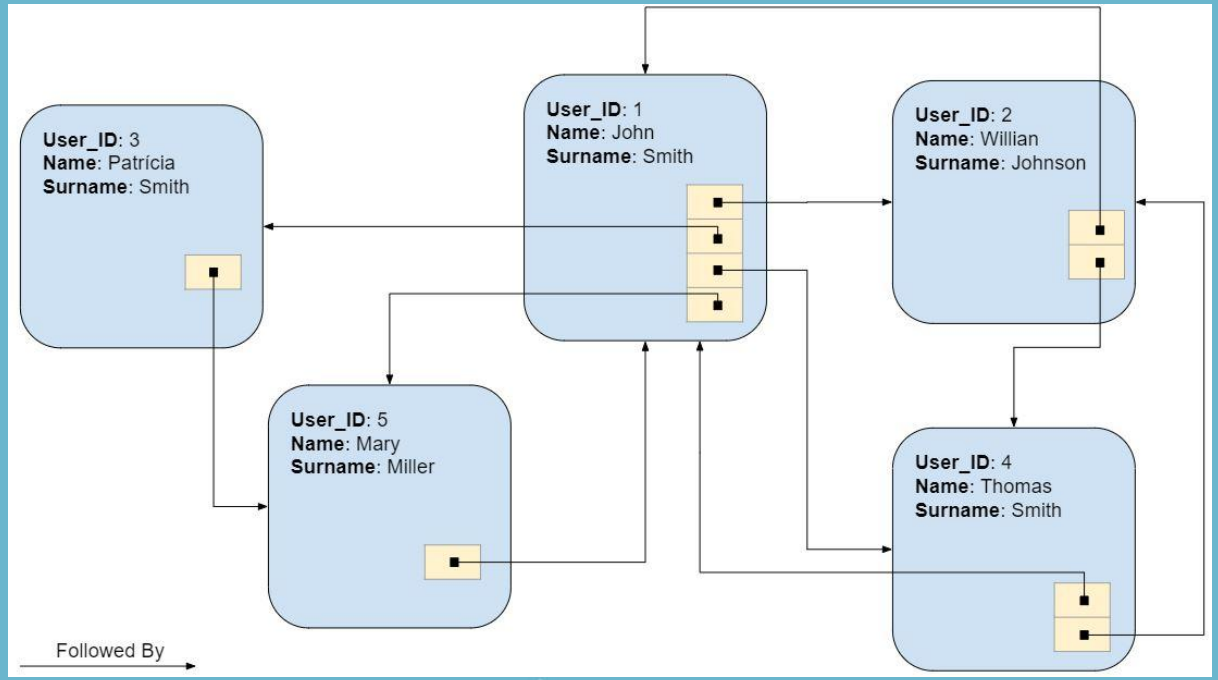
The underlying storage

- Some graph databases use **native graph storage** that is optimized and designed for storing and managing graphs.
- Not all graph database technologies use native graph storage, however. Some serialize the graph data into a relational database, an object-oriented database, or some other general-purpose data store.



The processing engine

A graph database use **index-free adjacency**, meaning that connected nodes physically “point” to each other in the database.

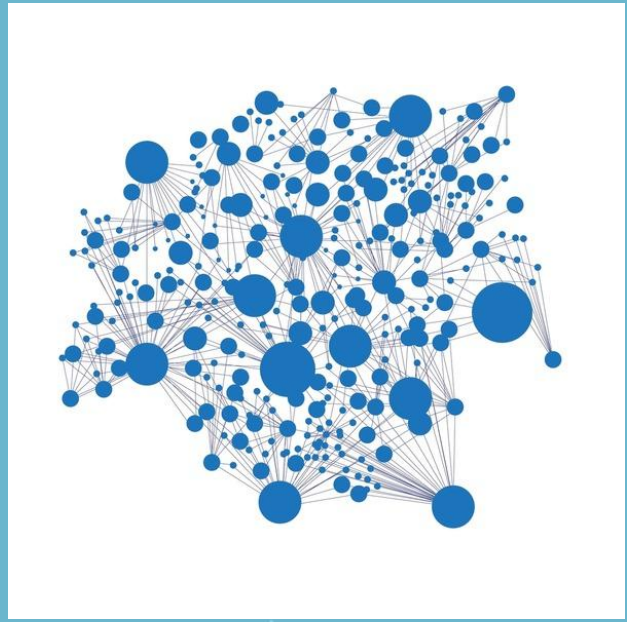


Each node directly references its adjacent nodes - each node actually holds a mini-index to all nearby nodes, making traversal operations extremely cheap.

The processing engine

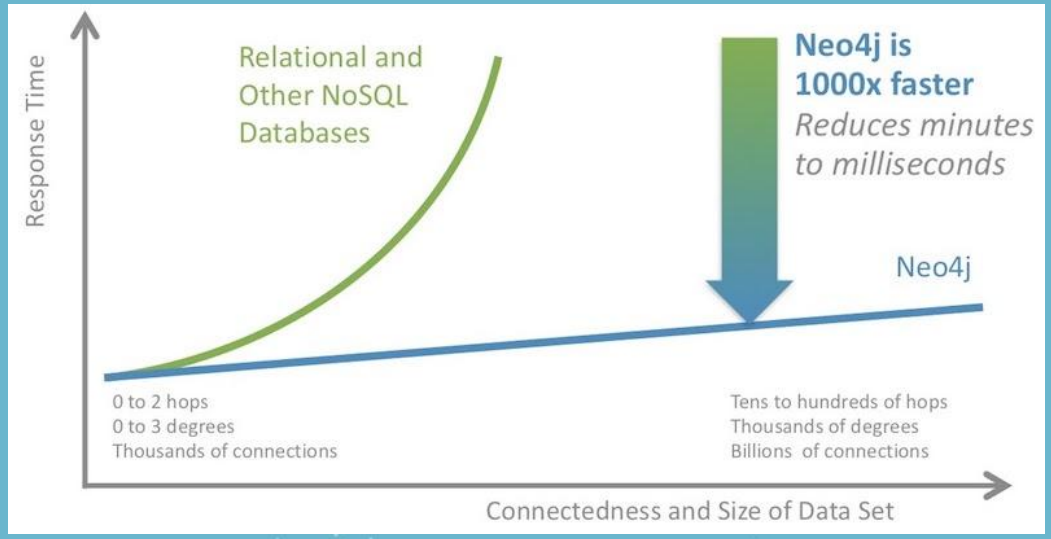
We take a slightly broader view:
*"any database that from the user's perspective **behaves** like a graph database (i.e., exposes a graph data model through CRUD operations) qualifies as a graph database"*

We acknowledge, however, the significant performance advantages of index-free adjacency, and therefore use the term ***native graph processing*** to describe graph databases that leverage index-free adjacency.



Why Choose Graph Databases?

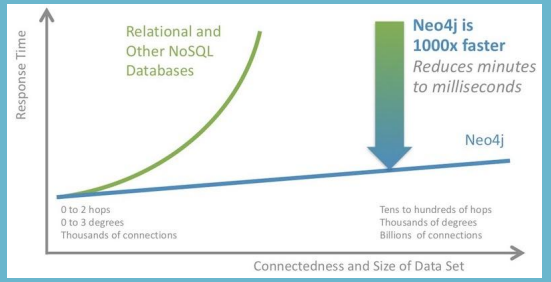
Powerful Performance
& Flexibility



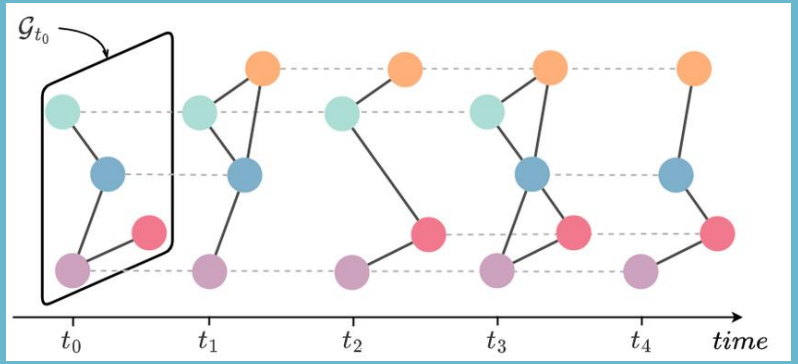
Why Choose Graph Databases?

Performance: Graph databases offer superior query performance for connected data, outperforming relational and NoSQL databases, especially **as data scales**. Unlike relational databases, where join-intensive queries slow down as datasets grow, **graph queries remain efficient and localized.**

Flexibility: The **schema-free** nature of graphs allows for adaptive data modeling, enabling developers to evolve the structure dynamically without extensive migrations or upfront schema definitions.



Why Choose Graph Databases?

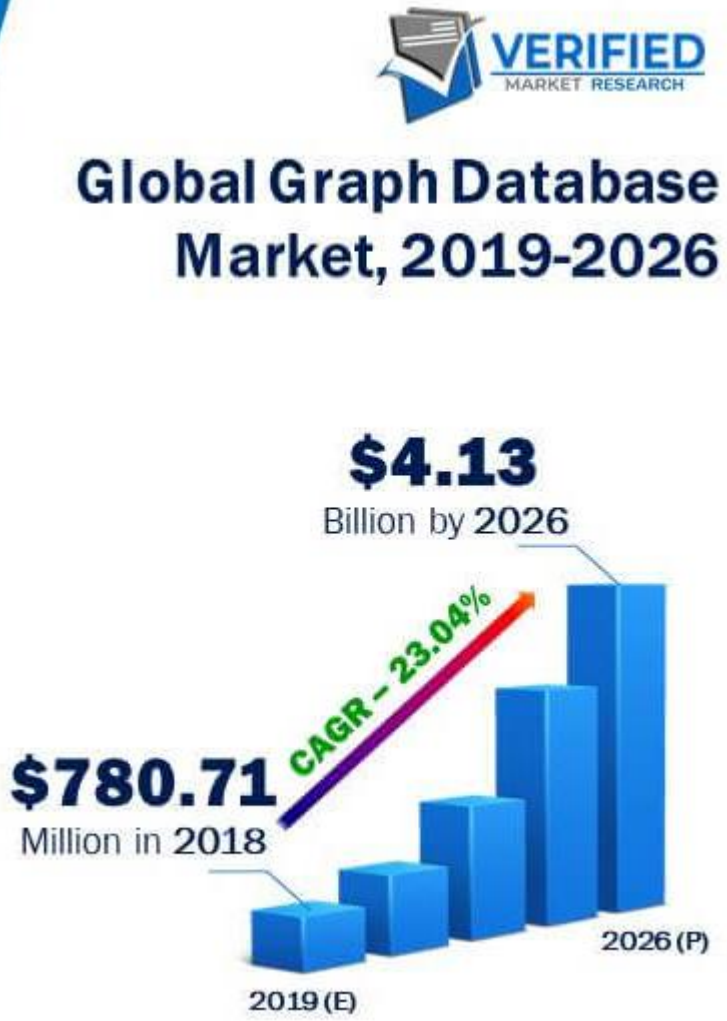


Agility & Evolving Data Models

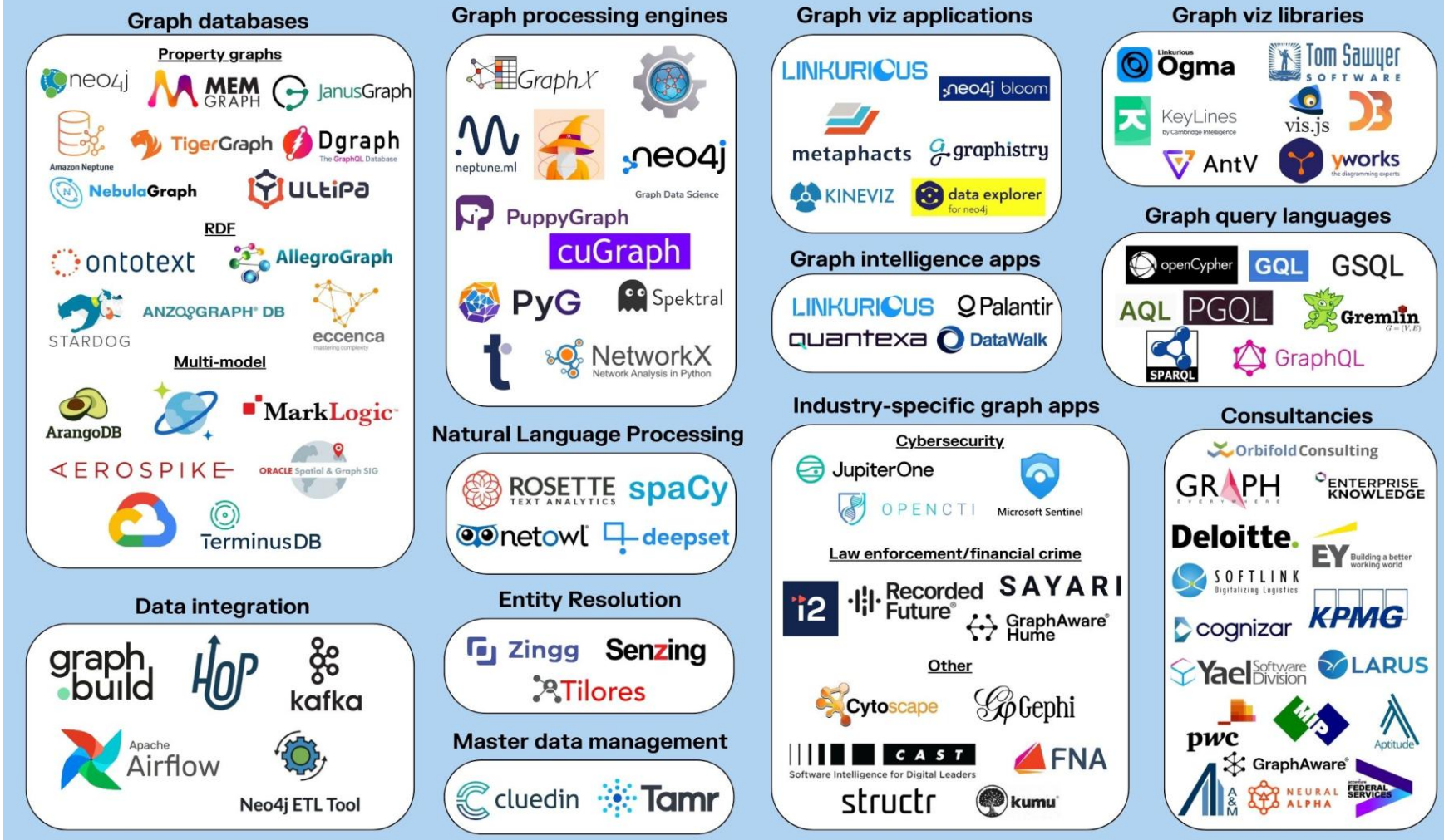
- Graph databases align with **agile software development**, supporting **iterative growth and seamless modifications**.
- Their **additive nature** allows adding relationships, nodes, and labels **without breaking existing queries**.
- Unlike relational databases, governance is applied **programmatically** through test-driven approaches, ensuring reliability in evolving applications.

Key Trends & Insights

- **Growing Adoption**: Businesses are integrating graph technology across various industries.
- **Graph & AI**: Knowledge graphs enhance large language models (LLMs) and other AI applications.
- **Expanding Market**: The graph ecosystem now includes tools for data ingestion, processing, and visualization, forming a comprehensive **graph data value chain**.



Graph technology landscape 2024



Key Takeaways from the Graph Technology Landscape

- The graph technology ecosystem has matured and diversified, covering everything from d atabases to visualization and industry applications.
- The rise of AI, cybersecurity, financial crime investigation, and NLP has significantly increas ed graph adoption.
- Graph databases and query languages like Neo4j, TigerGraph, Gremlin, Cypher, and SPARQ L remain fundamental in graph analytics.
- Visualization and intelligence applications are making graphs more accessible for business es and analysts.
- Large consulting firms (Deloitte, KPMG, EY) are investing in graph technology expertise, in dicating growing enterprise adoption.

Neo4j for OLTP (Transactional Workloads - Graph Database)

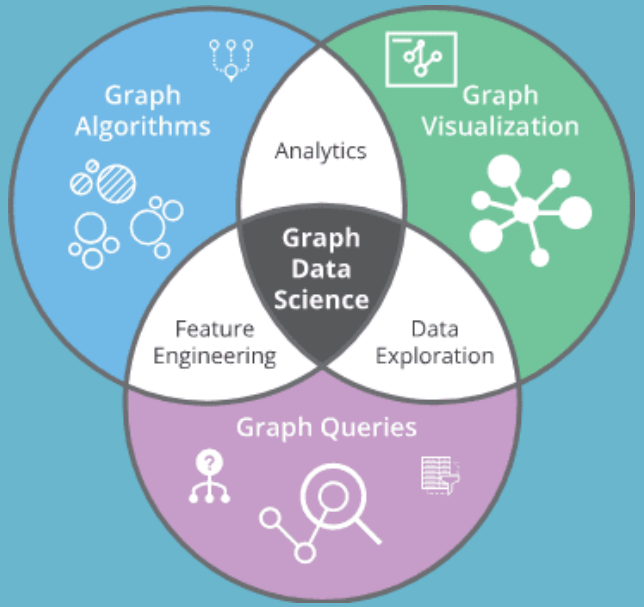
Neo4j is primarily designed as a **graph database**, meaning it excels in handling **real-time queries with frequent reads and updates**. → native graph db

- It supports **low-latency queries** using its Cypher query language.
- Ideal for applications requiring **fast traversal** of connected data, such as **recommendation engines, fraud detection, and social networks.**

Neo4j for OLAP (Graph Analytics - Compute Engine)

While Neo4j is **not optimized for large-scale batch processing**, it offers support for graph analytics through:

- **Graph Data Science (GDS) Library**
  - Provides advanced graph algorithms for community detection, centrality, and pathfinding.
  - Runs computations outside the main transactional engine for better performance.
- **APOC Procedures**
  - Extends Neo4j's capabilities with additional analytical functions.
- **Integration with Spark & Hadoop**
  - For large-scale graph analytics, Neo4j can export data to Apache Spark, GraphX, or Pregel-based frameworks for more efficient batch processing.



Limitations for OLAP in Neo4j:

- **Memory constraints**: Since Neo4j stores and processes graphs in memory, very large-scale analytics can be challenging.
- **Batch processing inefficiency**: Neo4j's transactional nature is not optimized for deep graph computations across massive datasets.

Thank you